
CRC
CREATIVE
ROBOTICS
CLUB

WELCOME TO THE

CREATIVE ROBOTICS CLUB

WHAT DO WE DO AT THE **CREATIVE ROBOTICS CLUB?**

We learn how to use electricity,
robotics and code to make things

We make art, design, or social robotics
- we support all disciplines

We reuse and repurpose where we can

We have fun

WE ARE OPEN TO YOUR FEEDBACK!

Are there things you want us to talk about?

A different way of running you think will work?

Skills you want to share?

We are a club for students, and we welcome your suggestions and input

BUT FIRST LETS TALK ABOUT...

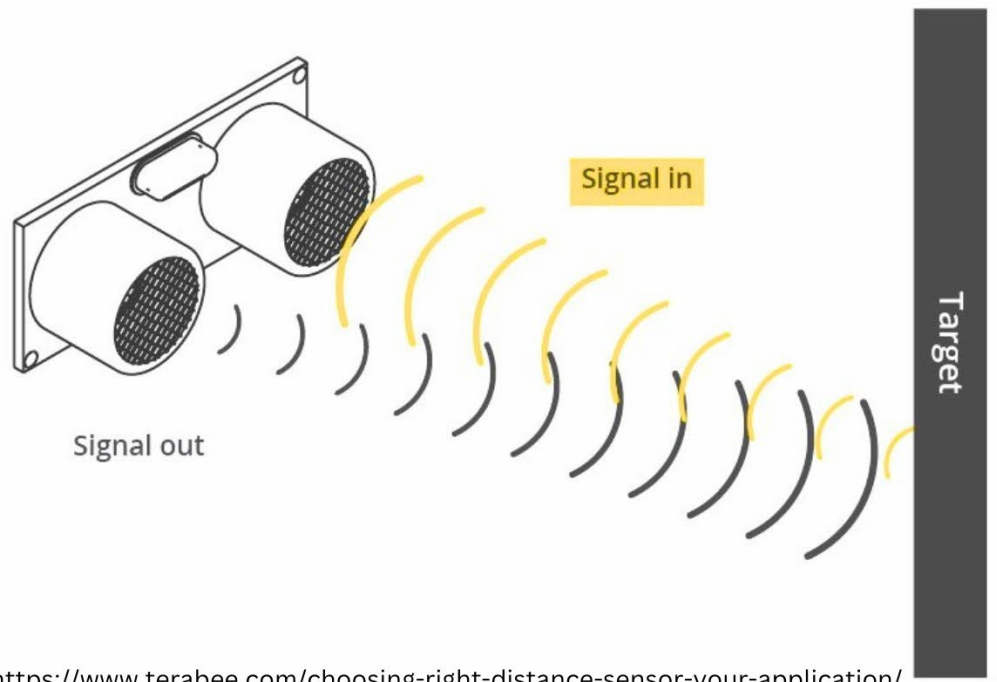
LAST WEEK

ULTRASONIC DISTANCE SENSOR

If we wanted to sense how far something was from our robot we might use an ultrasonic distance sensor.

This sensor sends out a burst of sound that is above human hearing and listens for the signal's return. The length of time between sending out the signal and hearing it lets it know just how far away an object in front of it is.

Plus it looks like eyes, and that's cute.



https://www.terabee.com/choosing-right-distance-sensor-your-application/

FUNCTIONS

So far we have written a lot of code to read sensors and make servos move, but it all gets crowded and it hard to tell apart. Currently, when we reuse code we're either copy and pasting it or typing it out again.

Is there a way we could make it easier to read and easier to reuse later?

void setup() and void loop() are called Functions, and we can write our own.

sketch_mar12a.ino

```
1  //We can use #define to set our pin variables,
2  //because we are never going to change them
3  #define trigPin 4
4  #define echoPin 5
5
6  void setup() {
7    Serial.begin(115200);
8    //Trig sends the signal
9    pinMode(trigPin, OUTPUT);
10   //Echo recieves the reflected signal
11   pinMode(echoPin, INPUT);
12 }
13
14 void loop() {
15   //Long is a type of variable that holds really big numbers
16   long duration, distance;
17   //Make sure the Trig pin is off
18   digitalWrite(trigPin, LOW);  // Added this line
19   delayMicroseconds(2); // Added this line
20   //Send the signal
21   digitalWrite(trigPin, HIGH);
22   delayMicroseconds(10); // Added this line
23   //Turn off the signal
24   digitalWrite(trigPin, LOW);
25   //Read the reflected signal
26   duration = pulseIn(echoPin, HIGH);
27   //Convert it to CM
28   distance = (duration/2) / 29.1;
29   Serial.print("Distance: ");
30   Serial.println(distance);
31 }
```

REVISITING IF{} STATEMENTS

Lets just focus on our
void loop() for now.

First we test:
if (distance > 200) {

We can use this to define our robots behaviour for all values over 200.

Next:
else if ()

Until now else has just been used as "in all other cases, do this instead" but we can add another if() statement there with else if()! If our first if statement is false, we can try again with a new test.

01-USDCode.ino

```
15  void loop() {
16    long distance = measureDistance();
17    distance = (lastDistance * 0.75) + (distance * 0.25);
18    lastDistance = distance;
19
20    if (distance > 200) {
21      // Code for if something is far away
22    } else if (distance < 200 && distance > 100) {
23      //Code for if something is a medium distance
24    } else if (distance < 100 && distance > 40) {
25      //Code if something is close
26    } else {
27      //Code if someting is REALLY close
28    }
29
30    Serial.print("Distance: "); Serial.println(distance);
31  }
32
33  long measureDistance()
34    {//Long is a type of variable that holds really big numb
35    long duration, result;
36    //Make sure the Trig pin is off
37    digitalWrite(trigPin, LOW);  // Added this line
38    delayMicroseconds(2); // Added this line
39    //Send the signal
40    digitalWrite(trigPin, HIGH);
41    delayMicroseconds(10); // Added this line
42    //Turn off the signal
```

REVISITING IF{} STATEMENTS

Ordering is important here.

We want to test only in ascending or descending values. This makes it easier to read and makes sure we're not accidentally catching the wrong values.

Also, we can only use an else after an if or an else if, but not in-between an if and an else if.

We write the code as:
if then else if then else.

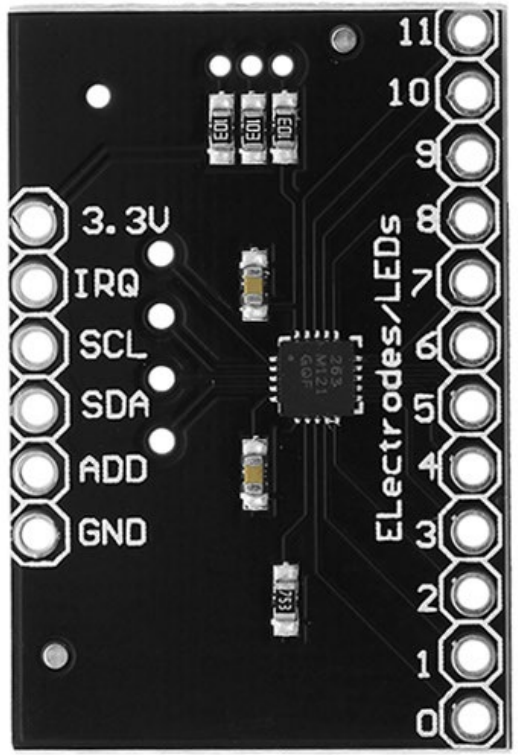
01-USDCode.ino

```
15  void loop() {
16    long distance = measureDistance();
17    distance = (lastDistance * 0.75) + (distance * 0.25);
18    lastDistance = distance;
19
20    if (distance > 200) {
21      // Code for if something is far away
22    } else if (distance < 200 && distance > 100) {
23      //Code for if something is a medium distance
24    } else if (distance < 100 && distance > 40) {
25      //Code if something is close
26    } else {
27      //Code if someting is REALLY close
28    }
29
30    Serial.print("Distance: "); Serial.println(distance);
31  }
32
33  long measureDistance()
34    {//Long is a type of variable that holds really big numb
35    long duration, result;
36    //Make sure the Trig pin is off
37    digitalWrite(trigPin, LOW);  // Added this line
38    delayMicroseconds(2); // Added this line
39    //Send the signal
40    digitalWrite(trigPin, HIGH);
41    delayMicroseconds(10); // Added this line
42    //Turn off the signal
```

Onto this week...

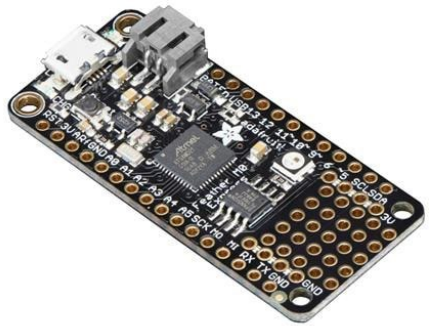
Slides available at:

www.github.com/creativeroboticsclub

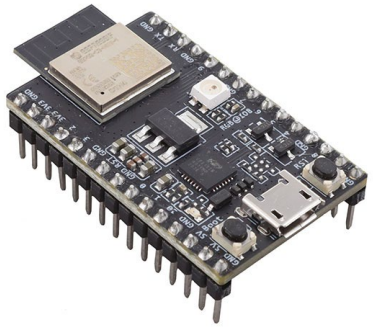


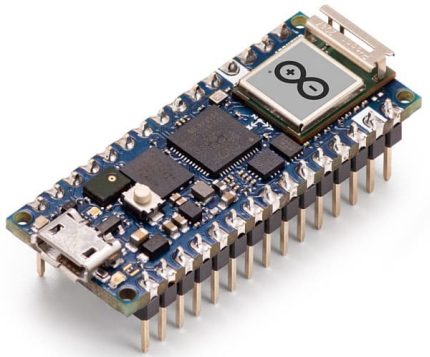
Moving on from Uno

Uno: a great way to learn, but maybe a bit big?



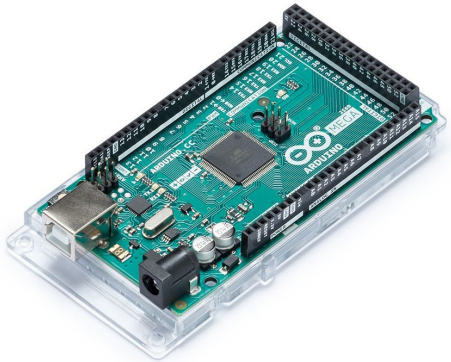
Arduino as a way of life (software, not just hardware)





Other microcontrollers exist?

Different applications, different requirements

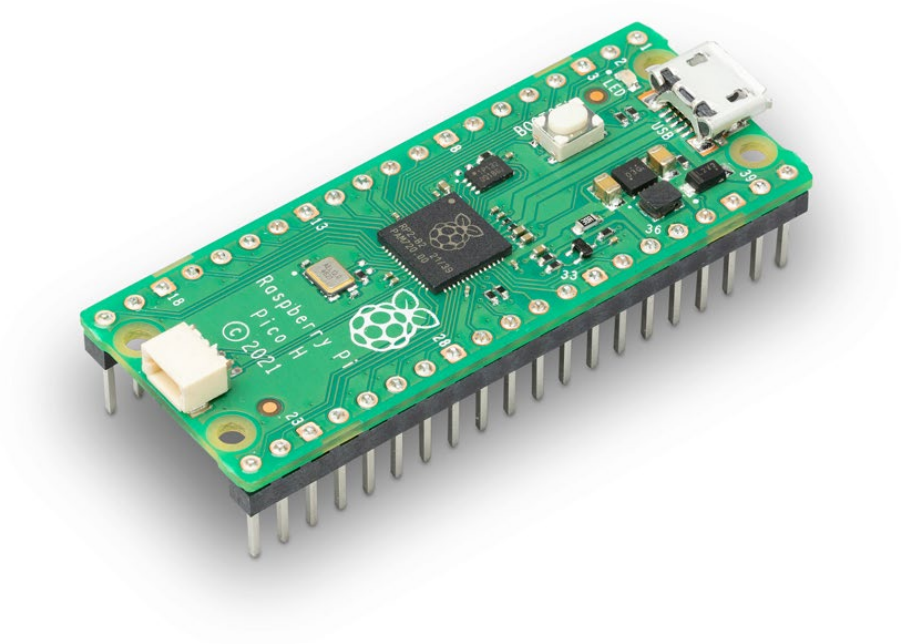


What is important for us?

Something small enough to fit
inside a small(ish) “body”

Arduino-compatible
(stick to what you know)

Cheap!

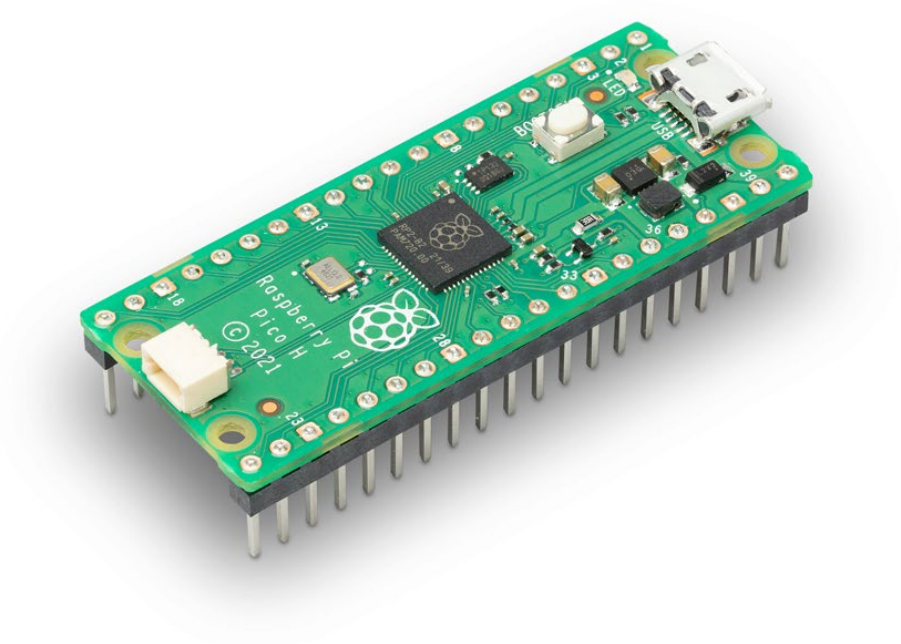


Raspberry Pi Pico

Made by the Raspberry Pi Foundation

Not to be confused with other
Pi boards (4B+, etc)

Available to buy for as little as $6



Wide compatibility:

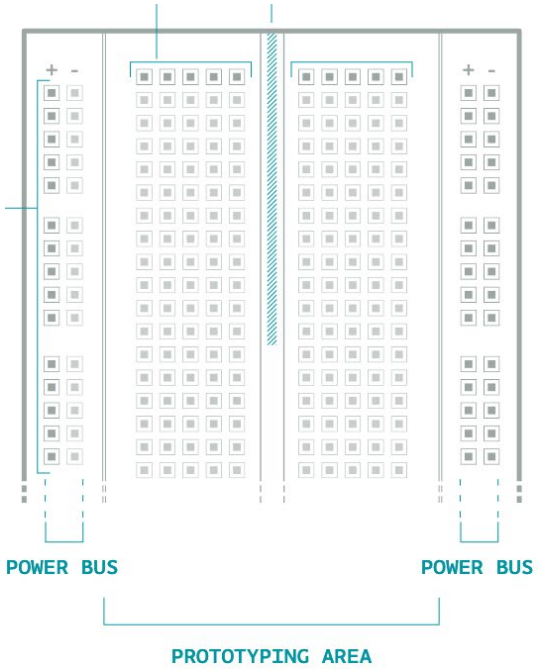
- Arduino
- CircuitPython
- MicroPython
- MakeCode

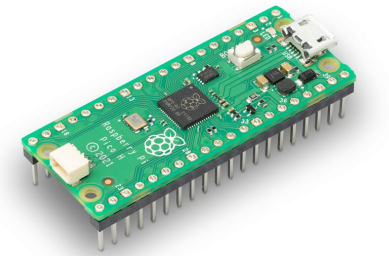
Breadboards

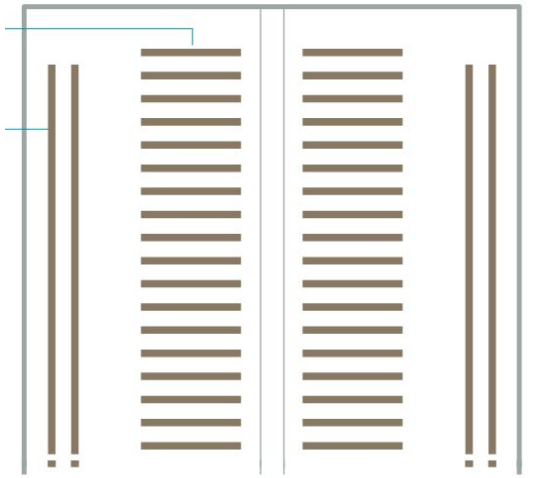
Allow us to make electrical connections without soldering

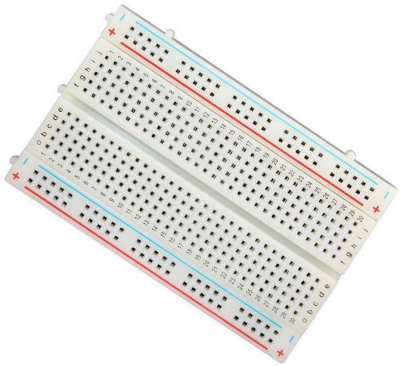
+ve and –ve to power bus (vertical connections)

Easy access to all pico pins (horizontal connections)









i²c

used by many sensors/actuators/shields

peripherals will require a **library**

multiple addresses means we can daisy-chain

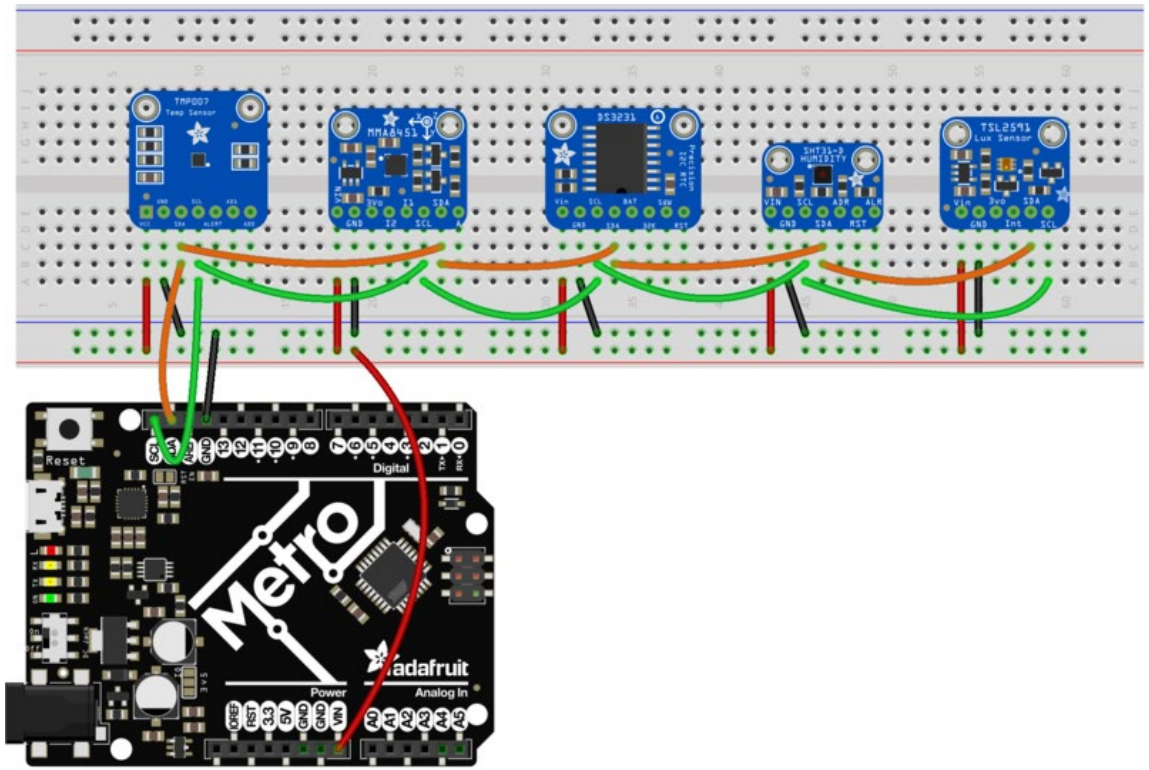
always requires 4 pins:

+

GND

SDA

SCL

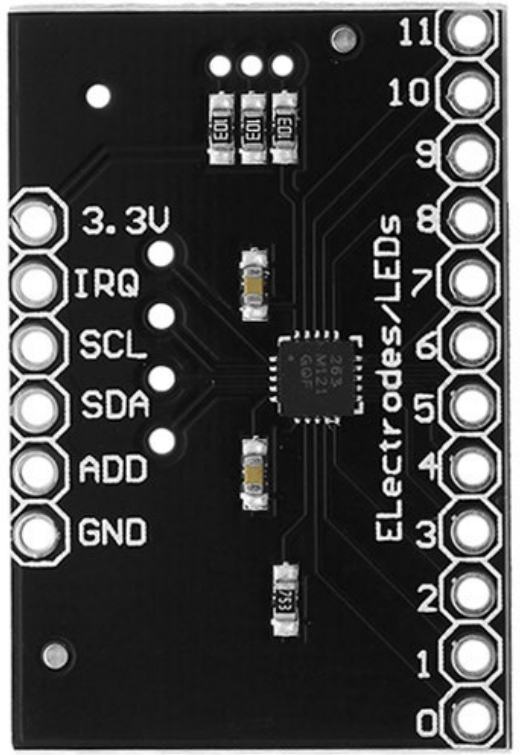


Capacitive touch

Simple way to use a variety of conductive materials as trigger sensors

MPR121: up to 12 touch pads without too much work

Requires the Adafruit_MPR121 library



Arrays

Store an ordered collection of data

Very specific declaration: type, name, [number of elements]

```
int myArray[3] = {1, 3, 5};
```

And to access the values inside the array:

```
Serial.println(myArray[0]);
Serial.println(myArray[1]);
Serial.println(myArray[2]);
```

Custom functions

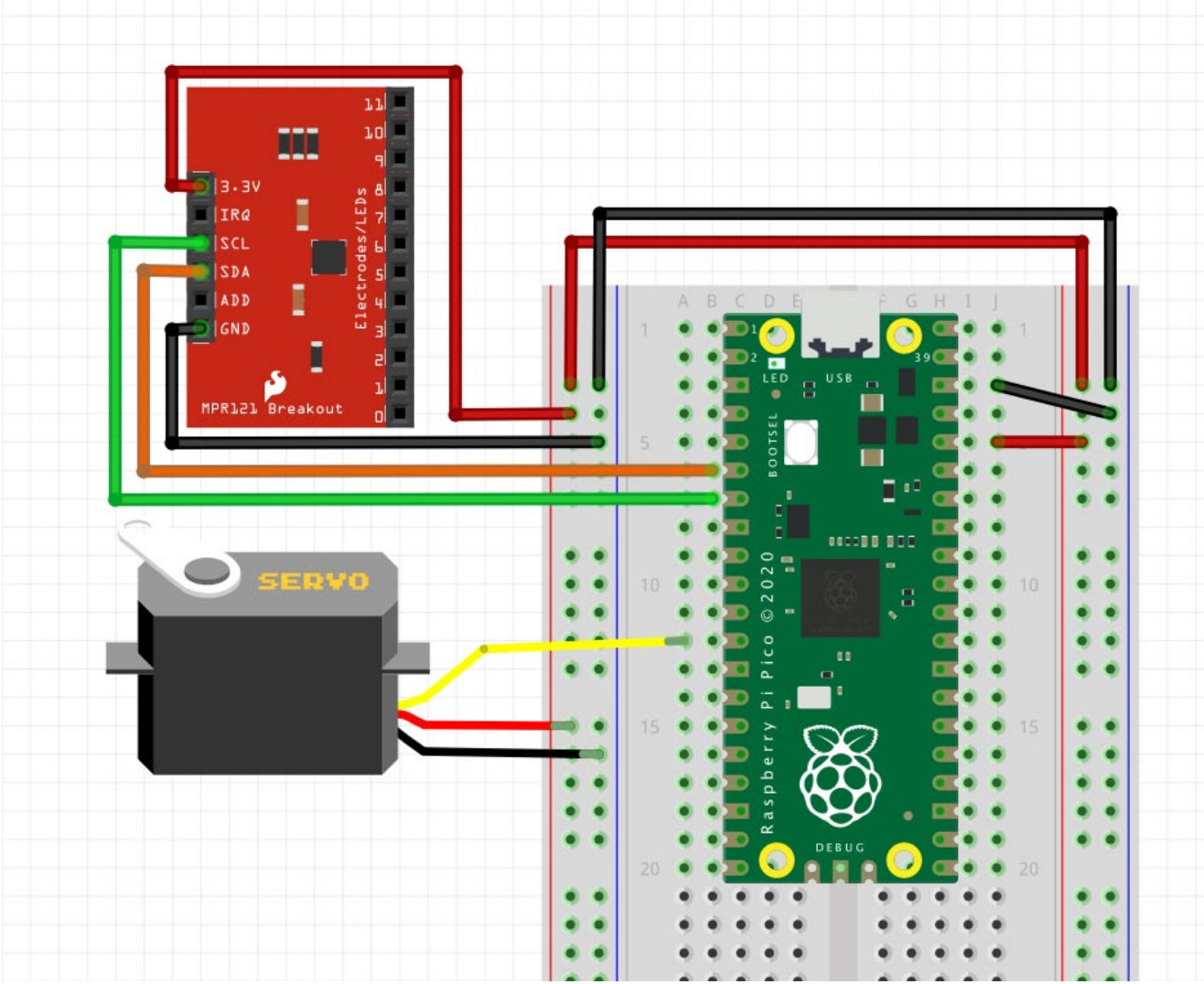
```
void setup(){
  // runs once
}

void loop(){
  // runs continuously
  myFunction();
}

void myFunction(){
  // runs when called
}
```

How do we connect our sensor breakout?

make sure +ve goes to 3.3V

note SDA and SCL location

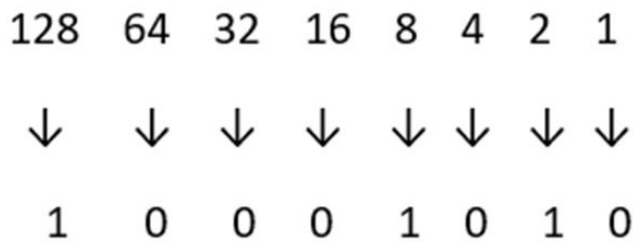
- Use the breadboard numbers to help you find pins 6 (SDA) and 7 (SCL)



And the code?

Some new concepts for us:

- Arrays[]
- Snapshot of touched pads is saved as an integer
- Binary representation of that int tells us which are on and off




```
1   // import our libraries
2   #include <Wire.h>
3   #include <Adafruit_MPR121.h>
4   #include <Servo.h>
5
6   #define shift 11 // used to convert from int to binary
7
8   Adafruit_MPR121 cap = Adafruit_MPR121(); // create our mpr121 object
9   Servo myservo;
10
11  int currtouched; // create integer to store our snapshot of touched pads
12  bool touchedPads[12] = { 0 }; // create an array of booleans, setting them all to 0
13
14  void setup() {
```

setup()

Begin Serial

Attach servo

Add a small part to check the connection to the MPR121

```
14  void setup() {
15    Serial.begin(115200); // begin Serial
16    myservo.attach(9); // attach our servo to pin 9
17
18    // if the Arduino can't find the MPR121, tell us
19    if (!cap.begin(0x5A)) {
20      Serial.println("MPR121 not found, check wiring?");
21      while (1);
22    }
23
24    // tell us when it's ready
25    Serial.println("MPR121 found!");
26  }
27
28  void loop() {
```

`loop()`

Take a snapshot of the currently touched pads

Revisiting if statements from last week: if the condition is TRUE or 1, run the first block
If the condition is FALSE or 0, run the `else` block

```
28  void loop() {
29    checkTouched(); // take a snapshot of the currently touched pads
30
31    // we use touchedPads[_] to check which pads are ON (1) or OFF (0)
32    if (touchedPads[0]) {
33      // Servo animation 1 goes here:
34      myservo.write(___);
35    } else {
36      // Servo "home" position goes here:
37      myservo.write(___);
38    }
39    // put a delay so it isn't overwhelming
40    delay(100);
41  }
```

Where the magic happens?

We know this looks confusing! You don't need to understand it all right now

We are converting an integer into binary representation and storing the 0s and 1s in our array of Booleans

```
43    void checkTouched() {
44      currtouched = cap.touched();
45      for (int i = shift; i >= 0; i--) {
46        byte bit = bitRead(currtouched, i);
47        touchedPads[i] = bit;
48      }
49    }
```

What about more than one pad at once?

All you need to do is update your if statement

Boolean logical operation reminders:

&& AND

|| OR

```
if(touchedPads[3] && touchedPads[9]){
  // Servo animation goes here:

} else if(touchedPads[0] || touchedPads[11]){
  // Other Servo animation goes here:

} else {
  // Servo "home" position goes here

}
```